Universidad de Alcalá

Máster Universitario en Ingeniería Electrónica

Guion de Laboratorio

Hito 1 — Adquisición y núcleo RTOS

Proyecto MIV-RT · Sesiones 3–7 · Entregable 1 (30 oct)

Última modificación: 19 de octubre de 2026

Asignatura:	Sistemas Electrónicos de Tiempo Real (202572)
Carácter:	Obligatoria · 3 ECTS · Aprendizaje Basado en Proyectos
Periodo:	1.er cuatrimestre · 12 sesiones + evaluación
Departamento:	Electrónica — Área de Tecnología Electrónica
Plataforma:	STM32F407G-DISC1 · FreeRTOS (CMSIS-V2) · STM32CubeIDE
Autor:	José Luis Martín Sánchez

Curso 2026/2027

Universidad de Alcalá

Hito 1

Adquisición y núcleo RTOS

Hito 1 · Sesiones 3–7 (5 – 30 oct 2026) · Entregable 1 (30 oct) · Teoría: Temas 2, 3 y 4 (I)

Objetivos de aprendizaje

Al terminar este hito, el estudiante será capaz de:

- **OL1.1** — Localizar en el esquemático y en el *datasheet* todo lo necesario para hablar con el acelerómetro: bus, pines, registros y protocolo SPI.
- **OL1.2** — Escribir un *driver* SPI propio del LIS3DSH y verificarlo con su registro de identificación.
- **OL1.3** — Implementar un *planificador cíclico* y explicar, midiendo, sus limitaciones (Tema 3).
- **OL1.4** — Implementar las tareas del modelo en FreeRTOS con prioridades *Rate Monotonic*, justificar la planificabilidad (Liu & Layland, RTA) y contrastarla con medidas.
- **OL1.5** — Calcular *pitch*, *roll* y el índice de vibración con la FPU e implementar la máquina de estados de seguridad del modelo.
- **OL1.6** — Reproducir una condición de carrera y protegerla con una sección crítica (Tema 4).

Idea clave del hito. El modelo dice *qué* hay que construir; este hito lo convierte en un sistema que **cumple sus plazos y lo demuestra**. Cada parte del guion aplica, esa misma semana, lo que se acaba de ver en teoría: primero un planificador hecho a mano (planificador cíclico), después un RTOS con prioridades, y en ambos casos **se predice con la teoría y se comprueba midiendo**.

1.1 Objetivo del hito y qué se entrega

Se parte del **Modelo de referencia del MIV-RT** (documento entregado con el Tema 2) y del entorno instalado en la sesión 2. Al cerrar el hito, la placa lee el acelerómetro cada 10 ms desde una tarea de FreeRTOS, calcula *pitch*, *roll* y el índice de vibración, y señaliza el estado de seguridad y la vibración con los LEDs según el modelo. Por el camino se habrán construido y medido dos ejemplos sencillos que ilustran la teoría: un planificador cíclico y un caso de sobrecarga.

Cómo leer este guion. Cada parte empieza con una caja **Teoría que se aplica**, que dice qué tema se está llevando a la placa. Lo que tiene que resolver el grupo está en cajas **Reto** numeradas; cada una indica **qué hay que conseguir** y **dónde buscar la información**.

Los fragmentos de código son *esqueletos*: las marcas `TODO` señalan lo que hay que escribir. La Tabla 1.3, al final, resume los retos y los relaciona con la rúbrica.

Qué se entrega (Entregable 1, 30 oct): el proyecto STM32CubeIDE funcionando y una memoria breve (PDF, 5–8 páginas) con la respuesta a los trece retos: datos con su fuente, cálculos, capturas o medidas, y la explicación de lo observado.

1.2 Conocimientos previos y material

- **Tema 1.** Criticidad *hard*/*firm*/*soft*, periodo, plazo, *jitter* y utilización $U = \sum C_i/T_i$.
- **Tema 2 y Modelo de referencia.** Requisitos RF-1 a RF-4, RF-6 y RNF-1, IBD, máquina de estados de seguridad y tabla de tareas. **Hay que tenerlo abierto durante todo el hito.**
- **Tema 3** (se imparte en paralelo, sesiones 4 y 5) y **Tema 4 (I)** (sesión 6).
- **Instrumental.** C con registros y máscaras de bits (`&`, `|`, `~`, `<<`, `>>`); lectura de un *datasheet* en inglés.

Material. Placa STM32F407G-DISC1 con su cable mini-USB (`CN1`); STM32CubeMX 6.17.0 y STM32CubeIDE 2.1.0 (instalados en la sesión 2). Para medir tiempos: osciloscopio o analizador lógico en un GPIO libre (se propone **PA8**) *o*, si no hay instrumento, el contador de ciclos DWT del procesador (Parte B). Documentación, que conviene descargar de st.com antes de la sesión 3: **UM1472** (manual y esquemático de la placa), **RM0090** (manual de referencia del STM32F407) y *datasheet* del **LIS3DSH**.

Punto de verificación

Punto de partida (sesión 2).

- ☐ El proyecto creado en CubeMX desde el *Board Selector* compila en CubeIDE y se carga en la placa.
- ☐ El LED verde LD4 (PD12) parpadea y el depurador se detiene en un punto de ruptura.

Si algo de esto falla, hay que resolverlo antes de la Parte A: el resto del hito lo da por hecho.

1.3 Parte A (sesión 3, 5 oct) — El sensor y su bus

Teoría que se aplica: Tema 2 · Del modelo a la implementación

En el IBD del modelo, la tarea Sensor recibe `MuestraAccel` del acelerómetro a través del puerto SPI del microcontrolador, y el modelo deja expresamente para este hito «pines, registros, modo SPI y *prescaler*». Esta parte concreta ese bloque del modelo: pasar de una caja del diagrama a un periférico configurado y un *driver* que funciona.

Paso A.1 — La conexión física. El acelerómetro está soldado en la placa y cuelga de uno de los periféricos SPI del microcontrolador. Antes de programar nada hay que saber cuál y por qué pines.

Tabla 1.1 – Conexión entre el STM32F407 y el LIS3DSH (a completar).

Señal	Pin MCU	Función (AF o GPIO)	Papel
SCK	*[completar]*	*[completar]*	Reloj de bus
MISO	*[completar]*	*[completar]*	Datos del sensor hacia el MCU
MOSI	*[completar]*	*[completar]*	Datos del MCU hacia el sensor
CS	*[completar]*	*[completar]*	Selección de chip

Reto 1.1 — Conexión física del sensor

1. Completar la Tabla 1.1 e indicar qué periférico SPI se usa.
2. La línea CS, ¿la gobierna el periférico SPI (NSS hardware) o un GPIO? ¿A qué nivel está activa? ¿Qué consecuencia tiene para el *driver*?

Qué hay que conseguir: la tabla completa, con la página de la que sale cada dato, y la respuesta sobre CS.

Dónde buscarlo: UM1472, esquemático del acelerómetro (*MEMS*) y tabla de asignación de pines de la placa. Contrastadlo con lo que muestra CubeMX en la vista *Pinout* del proyecto de la sesión 2.

Paso A.2 — Los registros del sensor. El LIS3DSH se maneja escribiendo y leyendo registros. Para el MIV-RT bastan unos pocos; se da la primera fila como ejemplo del nivel de detalle esperado.

Tabla 1.2 – Registros del LIS3DSH que usa el MIV-RT (a completar).

Registro	Dir. (hex)	Uso en el MIV-RT
`WHO_AM_I`	`0x0F`	Identificación, solo lectura: debe devolver `0x3F` (firma del LIS3DSH). Es la prueba de vida del bus.
`CTRL_REG4`	*[completar]*	*[completar]*
`CTRL_REG5`	*[completar]*	*[completar]*
`CTRL_REG6`	*[completar]*	*[completar]*
`OUT_X_L` … `OUT_Z_H`	*[completar]*	*[completar]*

Reto 1.2 — Configuración del sensor

1. Completar la Tabla 1.2, incluido el orden y el formato de los bytes de cada eje (con o sin signo, número de bits).
2. Elegir y justificar, **a partir de los requisitos del modelo**: la frecuencia de muestreo del sensor (ODR) frente al periodo de la tarea Sensor (RF-1); el fondo de escala (FS) para medir inclinación; y si conviene activar BDU (*Block Data Update*). Anotar la **sensibilidad** (mg/LSB) del FS elegido: se usa en la Parte B.
3. Calcular el **byte exacto** que hay que escribir en `CTRL_REG4` y en `CTRL_REG5`, mostrando de dónde sale cada bit.

Qué hay que conseguir: la tabla completa, tres decisiones justificadas y dos bytes en hexadecimal con su desglose.

Dónde buscarlo: *datasheet* del LIS3DSH: mapa de registros, descripción de `CTRL_REG4/5/6` y tabla de características mecánicas (sensibilidad).

Paso A.3 — El protocolo SPI. Cada transacción empieza con un byte que dice qué registro se quiere y si se lee o se escribe. Además, el sensor espera un modo de reloj concreto.

Reto 1.3 — Protocolo de lectura y escritura

1. ¿Cómo se distingue en el primer byte una lectura de una escritura? ¿Cuántos bytes viajan en total para leer los tres ejes de una vez, y qué permite hacerlo en una sola transacción?
2. Determinar el **modo SPI** (polaridad CPOL y fase CPHA) a partir del cronograma del *datasheet* y anotar dónde se configura en CubeMX.
3. Mientras el maestro envía el byte de dirección, también recibe un byte. ¿Qué contiene? ¿Qué implica para el *driver*?

Qué hay que conseguir: tres respuestas breves y razonadas; la del modo SPI se confirmará en placa en el Reto 1.5.

Dónde buscarlo: *datasheet* del LIS3DSH, sección de la interfaz SPI (protocolo de lectura y escritura y cronogramas).

Paso A.4 — El reloj del bus. En CubeMX (pestaña *Pinout & Configuration*, periférico SPI del sensor) se fija el modo y el *prescaler*. El reloj del bus es $f_{SCK} = f_{APB}/\text{prescaler}$, y una transacción de N bytes tarda $t_{SPI} = 8N/f_{SCK}$ (sin contar la sobrecarga del software). El *prescaler* es una decisión de diseño con dos límites: por arriba, el reloj máximo que admite el sensor; por abajo, el tiempo que la lectura resta a la tarea Sensor.

Reto 1.4 — Prescaler y tiempo de transacción

1. Obtener f_{APB} del bus del que cuelga el periférico y el f_{SCK} máximo del LIS3DSH. Comprobar qué *prescaler* deja CubeMX por defecto y si respeta ese límite.
2. Elegir el *prescaler* (¿cuál es el menor valor válido?) y calcular t_{SPI} para leer los tres ejes. Expresarlo como porcentaje del C estimado de Sensor en el modelo.
3. Con ese resultado, ¿es razonable leer por *polling* o haría falta DMA?

Qué hay que conseguir: el *prescaler* elegido y configurado, t_{SPI} y la conclusión sobre *polling*.

Dónde buscarlo: árbol de relojes de CubeMX (pestaña *Clock Configuration*) o RM0090; tabla de características SPI del *datasheet* del LIS3DSH; tabla de tareas del Modelo de referencia.

Paso A.5 — El *driver*. Se encapsula el acceso al sensor en tres funciones. Se dan el control de CS y las firmas; el cuerpo aplica lo resuelto en los Retos 1.2 y 1.3. En esta parte todavía no hay RTOS: se prueba desde el bucle de `main`.

```
#define LIS_READ   0x80   // bit de lectura en el primer byte
#define WHOAMI_REG 0x0F
#define WHOAMI_VAL 0x3F   // firma del LIS3DSH

```

```c
static void cs_low(void)  { HAL_GPIO_WritePin(/* TODO: puerto y pin de CS */, GPIO_PIN_RESET); }
static void cs_high(void) { HAL_GPIO_WritePin(/* TODO: puerto y pin de CS */, GPIO_PIN_SET); }

// Lee 'n' (<= 6) registros consecutivos a partir de 'reg'.
void lis_read(uint8_t reg, uint8_t *buf, uint16_t n)
{
    uint8_t tx[7] = {0}, rx[7];
    /* TODO: preparar tx[0] (direccion + bit de lectura)            */
    /* TODO: UNA transaccion full-duplex de n+1 bytes entre         */
    /*       cs_low() y cs_high(): HAL_SPI_TransmitReceive(...)     */
    /* TODO: copiar a 'buf' solo los bytes que son dato             */
}

void lis_write(uint8_t reg, uint8_t val)
{
    /* TODO: trama de 2 bytes para escritura                        */
}

// Devuelve 0 si el sensor responde y queda configurado; 1 si no.
uint8_t lis_init(void)
{
    /* TODO: prueba de vida con WHO_AM_I y, si pasa, escribir       */
    /*       CTRL_REG4 y CTRL_REG5 con los bytes del Reto 1.2       */
    return 1;
}
```

Listado 1.1 – Esqueleto del *driver* del LIS3DSH (CS es un GPIO).

¿Por qué una sola `HAL_SPI_TransmitReceive`? En SPI cada byte que sale implica uno que entra: mientras el maestro envía la dirección, recibe un byte sin significado que hay que descartar. Partir la lectura en `HAL_SPI_Transmit` + `HAL_SPI_Receive` también funciona con esta HAL (que limpia ese byte al terminar la primera llamada), pero oculta lo que ocurre en el cable y depende de ese detalle de la biblioteca. Una única transacción *full-duplex* de $n+1$ bytes lo hace explícito: el primer byte recibido se descarta y los n siguientes son el dato.

Reto 1.5 — Driver y prueba de vida

1. Completar el Listado 1.1 y llamarlo desde `main`: `WHO_AM_I` debe devolver `0x3F` de forma estable.
2. Tras escribir `CTRL_REG4` y `CTRL_REG5`, **leerlos de vuelta** y comprobar que contienen lo escrito.
3. Leer los seis bytes de salida en un bucle y observar en el depurador (*Live Expressions*) que cambian de forma coherente al inclinar la placa.
4. Si `WHO_AM_I` no dio `0x3F` a la primera, documentar el síntoma y la causa (modo SPI, CS, byte de dirección, primer byte recibido…).

Qué hay que conseguir: un *driver* propio que lee la firma y los tres ejes; evidencia (captura del depurador).

Dónde buscarlo: Retos 1.1 a 1.3; documentación de la HAL en CubeIDE (`HAL_SPI_TransmitReceive`, `HAL_GPIO_WritePin`).

Punto de verificación

- ☐ `WHO_AM_I` = `0x3F` de forma estable y la configuración se lee de vuelta correctamente.
- ☐ Los bytes de los tres ejes cambian al inclinar la placa.

1.4 Parte B (sesión 4, 19 oct) — Un planificador hecho a mano: el planificador cíclico

Teoría que se aplica: Tema 3 (I) · Planificador cíclico

La forma más simple de cumplir plazos sin sistema operativo es un **planificador cíclico** (*cyclic executive*): el tiempo se divide en marcos (*ciclo secundario*) de duración fija, y una tabla estática dice qué se ejecuta en cada marco; la tabla se repite cada *ciclo principal* (el mínimo común múltiplo de los periodos). Es predecible y fácil de analizar, pero frágil: una actividad que no cabe en su marco rompe toda la tabla. Esta parte lo construye y lo rompe a propósito.

Paso B.1 — De cuentas a unidades físicas. Cada eje se reconstruye como entero a partir de sus dos bytes (formato del Reto 1.2) y se convierte a mg con la sensibilidad anotada.

Paso B.2 — Ejemplo sencillo: un planificador cíclico para el MIV-RT. Antes de seguir, se guarda una copia del proyecto (por ejemplo, `MIVRT_ciclico`): en la Parte C se activará FreeRTOS y el planificador cíclico debe quedar como referencia para la memoria. Se implementan, sin RTOS, tres actividades del modelo con sus periodos: **leer el sensor** cada 10 ms, **convertir a mg** cada 20 ms y **conmutar un LED de latido** cada 50 ms.

```c
#define MARCO_MS   10
#define N_MARCOS   /* TODO: marcos del ciclo principal */
volatile uint32_t overruns = 0;          // marcos que no terminaron a tiempo

uint32_t t_marco = HAL_GetTick();
uint32_t k = 0;                          // numero de marco dentro del ciclo principal
while (1)
{
    while ((int32_t)(HAL_GetTick() - t_marco) < 0) { }  // espera al inicio del marco
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);  // traza: inicio de marco

    /* TODO: tabla de marcos: que actividades se ejecutan en el marco k */

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
    t_marco += MARCO_MS;
    if ((int32_t)(HAL_GetTick() - t_marco) > 0) overruns++;  // se ha pasado del marco
    k = (k + 1) % N_MARCOS;
}
```

Listado 1.2 – Esqueleto del planificador cíclico (marco de 10 ms, sin RTOS).

Medir sin osciloscopio: el contador DWT. El Cortex-M4 cuenta ciclos de CPU en `DWT->CYCCNT` (168 ciclos por microsegundo a 168 MHz). Se habilita una vez al arrancar con `CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; DWT->CYCCNT = 0; DWT->CTRL |=`

`DWT_CTRL_CYCCNTENA_Msk;`. La diferencia entre dos lecturas es una duración con resolución de ciclo, visible en *Live Expressions*. Se usa en todo el proyecto para medir periodos, tiempos de cómputo y bloqueos.

Reto 1.6 — Planificador cíclico: construirlo y romperlo

1. Calcular el ciclo principal y dibujar la **tabla de marcos** (qué actividad va en cada marco). Implementarla en el Listado 1.2 junto con la conversión a mg.
2. Medir el periodo de los marcos y su *jitter* (PA8 o DWT) y el tiempo de cada actividad.
3. Añadir, en un solo marco, una actividad «larga» de 15 ms de cómputo (una espera activa basta) y describir qué les ocurre a la lectura del sensor y al contador `overruns`.
4. ¿Cómo habría que reorganizar la tabla para que esa actividad cupiera sin romper el periodo de 10 ms? ¿Qué precio se paga? ¿Qué aportaría en cambio un planificador por prioridades con desalojo (Parte C)?

Qué hay que conseguir: un planificador cíclico que funciona, sus medidas, la evidencia del *overrun* y la reflexión final.

Dónde buscarlo: diapositivas del Tema 3 (planificador cíclico: ciclo principal y secundario); tabla de tareas del Modelo de referencia; Reto 1.2 (sensibilidad).

1.5 Parte C (sesión 5, 23 oct, 3 h) — FreeRTOS: las tareas del modelo con prioridades RM

Teoría que se aplica: Tema 3 (II y III) · Rate Monotonic, Liu & Layland y RTA

Con un planificador por prioridades fijas con desalojo, lo que decide si se cumplen los plazos es **cómo se asignan las prioridades**. Rate Monotonic (a menor periodo, mayor prioridad) es óptima entre las asignaciones fijas; el test de Liu & Layland da una condición suficiente y el análisis del tiempo de respuesta (RTA) una exacta. Esta parte lleva las tareas del modelo a FreeRTOS, aplica los dos tests y los pone a prueba con un ejemplo de sobrecarga.

Paso C.1 — Activar FreeRTOS. En CubeMX: *Middleware* → *FreeRTOS*, interfaz `CMSIS_V2`. En *Config parameters*, `configTICK_RATE_HZ` = **1000** y `configUSE_PREEMPTION` = 1. CubeMX pedirá que la base de tiempos de la HAL deje de ser SysTick: en *System Core* → *SYS* → *Timebase Source* se elige un temporizador libre (por ejemplo, `TIM6`).

Paso C.2 — Las tareas del modelo. Se crean los cuatro hilos de la tabla de tareas del modelo (*Tasks and Queues* en CubeMX, o por código). Comms queda con el cuerpo vacío hasta el Hito 3. CMSIS-V2 ofrece niveles de prioridad con nombre (`osPriorityLow` … `osPriorityHigh`, con subniveles `osPriorityLow1` … `Low7`, etc.).

Reto 1.7 — Prioridades y planificabilidad

1. Asignar las prioridades por RM y elegir sus niveles CMSIS. En el Hito 2 habrá que insertar **dos tareas más** por debajo de Sensor (Diagnóstico, de prioridad mínima, y otra de prueba, de prioridad intermedia): ¿deja hueco la asignación elegida sin alterar el orden de las cuatro?
2. Aplicar el test de Liu & Layland con los C estimados del modelo.
3. Medir el C real de cada tarea (DWT o traza) y repetir el test. ¿Cambia la conclusión? Actualizar la tabla de tareas con los valores medidos.

Qué hay que conseguir: tabla de prioridades con su justificación, el test con C estimados y con C medidos.

Dónde buscarlo: diapositivas del Tema 3 (RM y test de Liu & Layland); tabla de tareas del Modelo de referencia; documentación de CMSIS-RTOS2 (`osPriority_t`).

Paso C.3 — Periodicidad exacta. Un `osDelay(10)` al final del bucle espera 10 ms *después* de terminar el trabajo, así que el periodo real es $10 + C$ y varía con la carga. `osDelayUntil` despierta en instantes *absolutos* (Figura 1.1). Se parte de la versión ingenua:

```
void Task_Sensor(void *arg)
{
    uint8_t raw[6];
    for (;;) {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);   // traza: inicio
        lis_read(0x28, raw, 6);      // OUT_X_L..OUT_Z_H
        /* TODO: reconstruir los ejes y publicarlos para Proceso      */
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET); // traza: fin
        osDelay(10);                 // <- periodo NO exacto: corregir
    }
}
```

Listado 1.3 – Tarea Sensor con traza GPIO — versión *ingenua* de partida.

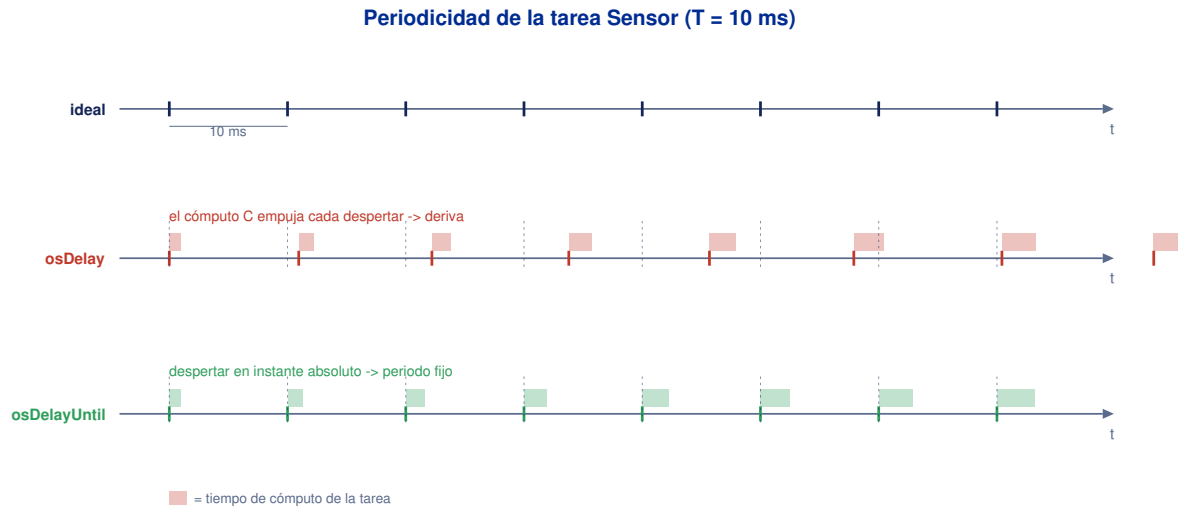


Figura 1.1 – Periodicidad de la tarea Sensor: `osDelay` acumula la deriva del cómputo; `osDelayUntil` mantiene el periodo fijo.

Reto 1.8 — Periodicidad medida

1. Medir periodo y *jitter* de Sensor con el Listado 1.3 tal cual, añadiendo dentro del bucle una carga artificial de $\sim$2 ms para que la diferencia se vea.
2. Corregirlo con `osDelayUntil`, repetir la medida en las mismas condiciones y explicar la diferencia con la Figura 1.1.
3. Comprobar que se cumple RF-1 y retirar la carga artificial.

Qué hay que conseguir: dos medidas comparables (captura o valores DWT) y la versión corregida.

Dónde buscarlo: diapositivas del Tema 3; documentación de CMSIS-RTOS2 (`osDelayUntil`, `osKernelGetTickCount`).

Paso C.4 — Ejemplo sencillo: sobrecarga y asignación de prioridades. Con los C del modelo, U es tan baja que cualquier asignación de prioridades funcionaría: no se aprende nada. Para ver por qué importa la asignación, se infla artificialmente el cómputo de Proceso hasta 12 ms de CPU, sin tocar su periodo. Para que la carga sea tiempo de CPU (y no de reloj, que seguiría corriendo mientras la tarea está desalojada) se usa el DWT:

```
// Consume 'us' microsegundos de CPU de LA PROPIA TAREA: si la tarea es desalojada,
// el tiempo que pasa fuera de la CPU no cuenta.
void quemar_cpu_us(uint32_t us)
{
    uint32_t acc = 0, prev = DWT->CYCCNT;
    while (acc < us * 168u) {
        uint32_t now = DWT->CYCCNT, d = now - prev; prev = now;
        if (d < 1680u) acc += d;          // saltos > 10 us = desalojo: no cuentan
    }
}
#define EJEMPLO_SOBRECARGA 1              // 0 en el sistema normal
```

Listado 1.4 – Carga sintética de CPU (solo para el ejemplo de sobrecarga).

Reto 1.9 — Sobrecarga: RM frente a otra asignación

1. Con Proceso a $C = 12$ ms, calcular U y aplicar el test de Liu & Layland. Calcular con RTA el tiempo de respuesta de peor caso de Sensor y de Proceso en dos casos: **(a)** prioridades RM; **(b)** Proceso por encima de Sensor.
2. Instrumentar Sensor para medir su **tiempo de respuesta máximo** (desde su instante de activación hasta el final de su trabajo) y **contar los plazos perdidos**.
3. Ejecutar los dos casos y comparar medidas y predicciones. ¿Qué requisito del modelo se incumple en el caso (b)? ¿Qué concluye el test de Liu & Layland en cada caso y por qué no basta?

Qué hay que conseguir: predicción (RTA) y medida de ambos casos, en una tabla, con la explicación.

Dónde buscarlo: diapositivas del Tema 3 (RTA: ecuación del tiempo de respuesta y su iteración); Retos 1.7 y 1.8.

1.6 Parte D (sesión 6, 26 oct) — Ángulos, seguridad y la primera condición de carrera

Teoría que se aplica: Tema 4 (I) · Sección crítica y condiciones de carrera

En cuanto dos tareas comparten un dato, el resultado puede depender del instante en que el planificador las desaloja: es una **condición de carrera**. La solución es convertir el acceso en una **sección crítica** indivisible. En el MIV-RT ya hay datos compartidos (los ejes y la actitud, en variables globales según el modelo); esta parte provoca una carrera a propósito para verla y la corrige con el mecanismo más básico.

Paso D.1 — Pitch y roll. Con la placa en reposo (solo actúa la gravedad), pitch $=\operatorname{atan2}\left(g_x, \sqrt{g_y^2+g_z^2}\right)$ y roll $=\operatorname{atan2}(g_y, g_z)$, en radianes. La FPU del Cortex-M4 es de **precisión simple**.

Reto 1.10 — Ángulos y FPU

1. Implementar el cálculo en grados en la tarea Proceso y validarlo con al menos tres posiciones de referencia (horizontal, de canto e inclinación intermedia medida con un transportador o una aplicación de nivel). Estimar el error frente a RF-2.
2. **Demostrar** que se usa la FPU: medir el tiempo del cálculo con `atan2f`/`sqrtf` y con `atan2`/`sqrt`, y comparar en el desensamblado las instrucciones que genera cada versión.

Qué hay que conseguir: ángulos validados con su error y la evidencia (medida + desensamblado) del uso de la FPU.

Dónde buscarlo: diapositivas del Tema 2 (tipos de dato `Actitud`); manual de referencia del Cortex-M4 (FPU de precisión simple); vista *Disassembly* de CubeIDE.

Paso D.2 — La máquina de seguridad. La tarea Seguridad implementa la máquina de estados del modelo (estados, umbrales con histéresis, LEDs y estado de FALLO en el arranque).

Reto 1.11 — Máquina de seguridad del modelo

Implementar la máquina de estados del Modelo de referencia (Tabla de estados y Figura 2.5) y demostrar en placa cada transición: OK↔AVISO↔PELIGRO, las transiciones directas OK↔PELIGRO (un giro brusco de la horizontal a más de $30°$ debe llegar a PELIGRO en una sola activación de Seguridad) y la entrada en FALLO si `lis_init` falla en el arranque (provocándolo de forma controlada, por ejemplo comparando con una firma falsa). Mostrar que la histéresis funciona: con la placa detenida junto a un umbral, el LED no oscila. Comprobar que el retardo cumple RF-3 también en el giro brusco y explicar de qué depende.

Qué hay que conseguir: la máquina completa funcionando y una prueba de cada transición.

Dónde buscarlo: Modelo de referencia (sección de comportamiento de seguridad); requisitos RF-3 y RF-4.

Paso D.3 — Ejemplo sencillo: una condición de carrera. Dos tareas incrementan un contador compartido: Sensor, una vez por activación, y una tarea de prueba `Task_Carrera` de **prioridad mínima** (por debajo de todas las del modelo) que lo hace en un bucle continuo, sin dormir. Cada tarea lleva además su propio contador *privado*, así que el valor correcto del compartido se conoce: la suma de los privados. El incremento se escribe a propósito en tres pasos (leer, «trabajar», escribir) para ensanchar la ventana en la que un desalojo hace daño.

> **¿Por qué no usar Proceso para esto?** Con prioridades RM y periodos armónicos (10 y 20 ms), Proceso siempre se ejecuta justo después de Sensor y termina mucho antes de la siguiente activación: Sensor nunca la interrumpe a mitad del incremento y la carrera no aparecería. Una tarea de fondo que siempre está lista garantiza que Sensor la desaloje una y otra vez en mitad de la ventana.

```
volatile uint32_t contador = 0;

volatile uint32_t privado_sensor = 0, privado_carrera = 0;

void incrementa(void)                 // la llaman Sensor y Task_Carrera
{
    uint32_t tmp = contador;          // 1) leer
    quemar_cpu_us(200);               // 2) "trabajo" entre la lectura y la escritura
    contador = tmp + 1;               // 3) escribir
}
```

Listado 1.5 – Contador compartido con una condición de carrera deliberada.

Reto 1.12 — Condición de carrera

1. Crear `Task_Carrera` (bucle continuo: `incrementa(); privado_carrera++;`) y añadir a Sensor `incrementa(); privado_sensor++;`. Tras unos segundos, comparar `contador` con `privado_sensor + privado_carrera`. Explicar con un cronograma cómo se pierde un incremento, de qué tarea es el incremento perdido y por qué la pérdida es aproximadamente proporcional al número de activaciones de Sensor.
2. Proteger la sección crítica con `taskENTER_CRITICAL()`/`taskEXIT_CRITICAL()` y comprobar que ya no se pierden incrementos. Medir el *jitter* de Sensor con y sin la protección: ¿qué precio se paga? ¿Por qué no es buena idea usar este mecanismo para secciones críticas largas? (El Tema 4 (II) y el Hito 2 dan la alternativa.)
3. Aplicarlo al MIV-RT: Sensor publica tres ejes en tres variables globales. ¿Es atómica cada escritura en un Cortex-M4? ¿Y el trío? ¿Puede Proceso combinar ejes de dos muestras distintas? Decidir si importa y, si importa, protegerlo.

Qué hay que conseguir: la carrera reproducida y explicada, la corrección y el análisis del trío de ejes.

Dónde buscarlo: diapositivas del Tema 4 (condición de carrera, sección crítica, inhibición de interrupciones); documentación de FreeRTOS (*critical sections*).

1.7 Parte E (sesión 7, 30 oct, 3 h) — Vibración, integración y entrega

Teoría que se aplica: Tema 3 · Tema 4 (I) · Qué muestras ve cada tarea

Añadir cálculo a una tarea cambia su C, y el análisis de planificabilidad hay que **rehacerlo con el valor medido**, no darlo por supuesto. Además, el mecanismo de comunicación decide qué datos llegan: una variable global es un **estado** (el último valor escrito), no una secuencia de **eventos**. Si el productor escribe más deprisa de lo que el consumidor lee, los valores intermedios se pierden sin que nadie se entere. El índice de vibración lo hace visible.

Paso E.1 — Índice de vibración (RF-6). El Modelo de referencia define V como la desviación típica del módulo de la aceleración, $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$, en una ventana de W muestras, y lo señaliza con el LED azul (LD6) con dos umbrales, V_{on} y V_{off}. El modelo no fija W ni los umbrales: son decisiones del grupo. En este hito, Proceso calcula V con las muestras que le llegan por las variables globales de Sensor.

Reto 1.13 — Índice de vibración

1. Implementar V en Proceso y el LED azul con histéresis. Elegir W para que se cumpla el retardo de RF-6 y justificarlo. ¿Por qué el modelo usa el módulo $|a|$ y no un eje? Comprobarlo: inclinar despacio la placa no debe encender el LED azul.
2. Caracterizar V en placa: con la placa quieta sobre la mesa (el ruido del sensor) y con al menos dos excitaciones reproducibles (golpes en la mesa, agitarla con la mano…). Con esas medidas, elegir V_{on} y V_{off} con un margen justificado.
3. Añadir un número de secuencia a cada muestra en Sensor y contar desde Proceso cuántas muestras de cada ventana se usan de verdad. Explicar el resultado con los periodos y las prioridades del Reto 1.7. ¿Qué frecuencias de vibración puede ver aún V con las muestras que llegan? RF-6 pide la ventana completa: se resuelve en el Hito 2.
4. Medir el C de Proceso con el cálculo de V y rehacer el test de planificabilidad (RNF-1). ¿Cambia la conclusión del Reto 1.7?

Qué hay que conseguir: V funcionando con el LED azul; W, V_{on} y V_{off} justificados con medidas; las muestras perdidas contadas y explicadas; el análisis de planificabilidad actualizado.

Dónde buscarlo: Modelo de referencia (RF-6 e indicador de vibración); *datasheet* del LIS3DSH (ODR, ruido, filtro antisolapamiento); diapositivas del Tema 3 (Liu & Layland, RTA) y del Tema 4 (I) (datos compartidos); Retos 1.7 y 1.12.

Paso E.2 — Integración. Con los ejemplos de sobrecarga y de carrera **desactivados** (`#define` a 0), el sistema debe quedar como lo describe el modelo para este hito: Sensor, Proceso y Seguridad con sus periodos y prioridades, y Comms creada pero vacía. La sesión se dedica a integrar, repetir las medidas finales y cerrar la memoria. Es también el **punto de control**: el profesor revisa con cada grupo el estado del sistema antes de empezar el Hito 2.

Punto de verificación

- ☐ El periodo de Sensor es $10 \pm 0{,}5$ ms (RF-1) con el sistema completo.
- ☐ *Pitch* y *roll* coinciden con la inclinación real (RF-2).
- ☐ Los LEDs siguen la máquina de estados del modelo y no oscilan en los umbrales (RF-3); un fallo del sensor en el arranque lleva a FALLO (RF-4).
- ☐ El LED azul se enciende al golpear o agitar la placa y no al inclinarla despacio (RF-6, salvo la ventana completa).
- ☐ La tabla de tareas tiene los C medidos y el test de planificabilidad está hecho con ellos (RNF-1).

1.8 Entregable y criterios de evaluación

Entregable 1 (30 oct): el proyecto STM32CubeIDE (también la copia `MIVRT_ciclico`) y una memoria breve con la respuesta a los retos, en su orden. El Entregable 1 pesa un **30 %** de la nota práctica.

Tabla 1.3 – Qué resuelve el grupo en el Hito 1.

Reto	Tema	Resultado esperado	Criterio
1.1	T2	Conexión SPI con fuentes; papel de CS	Arquitectura
1.2	T2	Registros; ODR, FS y BDU justificados; bytes de configuración	Funcionalidad
1.3	T2	Bit de lectura, autoincremento, modo SPI, primer byte	Funcionalidad
1.4	T2	*Prescaler* dentro del límite del sensor; t_{SPI}	Calidad TR
1.5	T2	*Driver* propio; firma, relectura y ejes	Funcionalidad
1.6	T3 (I)	Planificador cíclico: tabla, medidas, *overrun* y reflexión	Calidad TR
1.7	T3 (II)	Prioridades RM con hueco; Liu & Layland con C estimado y medido	Calidad TR
1.8	T3	`osDelay` frente a `osDelayUntil`, medidos	Calidad TR
1.9	T3 (III)	Sobrecarga: RTA y medida con RM y con otra asignación	Calidad TR
1.10	T2	Ángulos validados; FPU demostrada	Funcionalidad
1.11	T2	Máquina de seguridad del modelo, todas las transiciones	Funcionalidad
1.12	T4 (I)	Carrera reproducida y corregida; análisis del trío de ejes	Arquitectura
1.13	T3 · T4 (I)	V con LED azul; W y umbrales medidos; muestras perdidas explicadas; planificabilidad actualizada	Funcionalidad

1.9 Errores frecuentes y consejos

- **`WHO_AM_I` devuelve `0xFF` o `0x00`.** Sospechosos habituales, por este orden: CS no baja (o lo gobierna el periférico), modo SPI distinto del que espera el sensor, byte de dirección mal formado y descarte del primer byte recibido.
- **Datos sin fuente.** Un pin o una dirección de registro sin la página de la que sale no se puede revisar.
- ***Prescaler* «para ir más rápido».** Por encima del reloj máximo del sensor, el sistema queda fuera de especificación aunque funcione en una placa concreta.

Tabla 1.4 – Rúbrica del Hito 1.

Criterio	Peso	Qué se valora
Funcionalidad	40 %	*Driver* y configuración del sensor correctos; ángulos validados con su error; máquina de seguridad del modelo completa (FALLO incluido); indicador de vibración con umbrales basados en medidas.
Calidad de tiempo real	30 %	Planificador cíclico medido y su límite explicado; periodicidad exacta demostrada; prioridades RM justificadas con Liu & Layland y RTA; predicciones contrastadas con medidas en el ejemplo de sobrecarga.
Arquitectura y código	20 %	Datos con su fuente; *driver* limpio y reutilizable; FPU demostrada; carrera explicada y corregida; código organizado por tareas.
Integración	10 %	Sistema listo para el Hito 2: Comms creada, `Actitud` disponible y ejemplos desactivables por `#define`.

- **Perder el planificador cíclico.** Si no se guarda la copia antes de activar FreeRTOS, el Reto 1.6 se queda sin evidencia.
- `osDelay` **en lugar de** `osDelayUntil`**.** Si el periodo «respira» con la carga, es esto.
- **Carga sintética con el reloj.** Una espera `while (tick - t0 < 12)` no es tiempo de CPU: si la tarea es desalojada, el reloj sigue corriendo y la carga real es menor. Por eso se da `quemar_cpu_us`.
- **Funciones** `double` **sin querer.** `atan2` sin la `f`, o constantes como `0.06` sin `f`, fuerzan cálculo en doble precisión por software.
- **Olvidar desactivar los ejemplos.** El sistema entregado debe cumplir RF-1 a RF-4 y el indicador de RF-6: la sobrecarga y la carrera van tras un `#define` a 0.